

0. Setup — paths and libraries

```
In [ ]: # =====
# PATHS (edit these two lines to match your machine)
# =====
DATA_PATH = r"C:\Users\verma\Desktop\Code_PC\New data\Input and Output 16 Sep.xl"
OUTPUT_DIR = r"C:\Users\verma\Desktop\Code_PC\New data\Bo_Ca_Re_We_V.R_D.R1"

import os
os.makedirs(OUTPUT_DIR, exist_ok=True)
print("Feature set      :", ["Bo", "Ca", "Re", "We", "V.R", "D.R"])
print("Plots will be saved to:", OUTPUT_DIR)
```

```
In [ ]: # LIBRARIES
# -----
# Numerical and data handling
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
import logging
logging.getLogger("matplotlib.font_manager").setLevel(logging.ERROR) # silence

# Visualization
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

# Statistical analysis
from scipy.stats import linregress

# Machine Learning models
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor
from catboost import CatBoostRegressor

# Evaluation metrics (imported once, no duplicates)
from sklearn.metrics import (
    r2_score,
    mean_squared_error,
    mean_absolute_error,
    mean_absolute_percentage_error
)

# Explainable AI
import shap
```

1. Load & inspect the data

```
In [ ]: ## Read the data
data = pd.read_excel(DATA_PATH)
```

```
print("Shape:", data.shape)
data.head()
```

```
In [ ]: # Inspect the dataset
print(data.head())
print(data.info())
print(data.describe())
```

```
In [ ]: # Check for missing values
print("\nMissing Values:")
print(data.isnull().sum())

# Remove rows with missing values
data = data.dropna()
data.shape
```

```
In [ ]: # Check for duplicate records
print("\nDuplicate Rows:", data.duplicated().sum())

# Remove duplicates
data = data.drop_duplicates()
data.shape
```

```
In [ ]: # Descriptive statistics
stats_table = data.describe().T

# Add variance and coefficient of variation
stats_table["Variance"] = data.var()
stats_table["CV (%)"] = (data.std() / data.mean()) * 100

# Round values
stats_table = stats_table.round(2)

# Create figure
fig, ax = plt.subplots(figsize=(14, 7))
ax.axis("off")

table = ax.table(
    cellText=stats_table.values,
    rowLabels=stats_table.index,
    colLabels=stats_table.columns,
    cellLoc="center",
    rowLoc="center",
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.7)

for (row, col), cell in table.get_celld().items():
    cell.set_linewidth(0.8)
    if row == 0:
        cell.set_text_props(weight="bold")

plt.title("Descriptive Statistics", fontsize=14, fontweight="bold", pad=20)
plt.tight_layout()

plt.savefig(
```

```

os.path.join(OUTPUT_DIR, "Descriptive_Statistics_Table.png"),
dpi=600,
bbox_inches="tight"
)

plt.show()

```

2. Exploratory plots (this variant's features + target)

`features_all` here = this variant's **training features** (Bo, Ca, Re, We, V.R, D.R) **plus the target** (δ/r), so the violin plot and correlation matrix always show exactly what this run is modeling. The violin-plot grid (rows $\times$ cols) is computed automatically from the number of panels needed, so it adapts whether there are 3, 4, or 7 panels (2 features / 3 features / 6 features, each plus target), instead of the previous notebook's fixed 2 $\times$ 3 grid.

```

In [ ]: sns.set(style="whitegrid", font_scale=1.1)

features_all = ["Bo", "Ca", "Re", "We", "V.R", "D.R"] + [' $\delta/r$ ']
palette = sns.color_palette("Set2", len(features_all))

# Dynamic grid: fixed at 3 columns, as many rows as needed for len(features_all)
ncols = 3
nrows = int(np.ceil(len(features_all) / ncols))

fig = plt.figure(figsize=(14, 4 * nrows), dpi=600)

for i, col in enumerate(features_all):

    ax = plt.subplot(nrows, ncols, i + 1)

    sns.violinplot(y=data[col], ax=ax, color=palette[i], inner=None, linewidth=1)

    sns.boxplot(
        y=data[col], ax=ax, width=0.15,
        boxprops=dict(facecolor="white", edgecolor="black"),
        medianprops=dict(color="red", linewidth=2),
        showfliers=False
    )

    sns.stripplot(y=data[col], ax=ax, color="black", size=2, jitter=0.25, alpha=

    ax.set_title(col, fontsize=12, fontweight='bold')
    ax.set_ylabel("Value")

    ax.text(0.02, 0.95, f"({chr(97+i)})", transform=ax.transAxes, fontsize=12, f

    if col == 'Re':
        ax.set_yscale('log')

    sns.despine(ax=ax)

save_path = os.path.join(OUTPUT_DIR, "violin_plot.png")
plt.savefig(save_path, dpi=600, bbox_inches="tight", facecolor="white")
plt.show()

```

```
print(f"Figure saved to: {save_path}")
```

```
In [ ]: corr = data[features_all].corr()

fig, ax = plt.subplots(figsize=(8, 6), dpi=600)

sns.heatmap(
    corr, annot=True, cmap="coolwarm", fmt=".2f",
    linewidths=0.5, linecolor='black', vmin=-1, vmax=1, square=True,
    cbar_kws={"shrink": 0.8}, annot_kws={"size": 10, "weight": "bold"}, ax=ax
)

ax.set_title("Correlation Matrix", fontsize=12, fontweight='bold')
fig.tight_layout(rect=[0, 0, 1, 0.95])

save_path = os.path.join(OUTPUT_DIR, "correlation_matrix.png")
fig.savefig(save_path, dpi=600, bbox_inches="tight", facecolor="white")

print(corr)
plt.show()
```

3. Features / target, split, scaling

This variant's training features: Bo, Ca, Re, We, V.R, D.R. `X` is built as an explicit column selection, so only these columns are ever used to train the models below (any other columns present in the spreadsheet are ignored for this run).

```
In [ ]: # Define features (X) and target (y) for THIS variant.
feature_cols = ["Bo", "Ca", "Re", "We", "V.R", "D.R"]
X = data[feature_cols].copy()
y = data["S/r"].copy()

print("Feature matrix shape:", X.shape)
print("Target shape          :", y.shape)
print("Training features    :", list(X.columns))

excluded = [c for c in data.columns if c not in feature_cols and c != "S/r"]
if excluded:
    print("Excluded columns      :", excluded, "(present in file, not used as mo
```

```
In [ ]: # Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print('Training shape:', X_train.shape)
print('Testing shape  :', X_test.shape)
```

```
In [ ]: # Standardize features (fit on train, apply to both)
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns, index=X_test.index)
```

```
# Use the scaled versions from here on
X_train = X_train_scaled
X_test = X_test_scaled

print("Training shape:", X_train.shape)
print("Testing shape :", X_test.shape)
```

4. Plot style & helper functions

```
In [ ]: # =====
# GLOBAL PLOT STYLE
# =====

plt.style.use('default')

mpl.rcParams['font.family'] = 'Times New Roman'
mpl.rcParams['font.weight'] = 'bold'
mpl.rcParams['axes.labelweight'] = 'bold'
mpl.rcParams['axes.titleweight'] = 'bold'
mpl.rcParams['axes.edgecolor'] = 'black'
mpl.rcParams['axes.linewidth'] = 2.0
mpl.rcParams['xtick.color'] = 'black'
mpl.rcParams['ytick.color'] = 'black'
mpl.rcParams['xtick.major.width'] = 2.0
mpl.rcParams['ytick.major.width'] = 2.0
mpl.rcParams['text.color'] = 'black'
mpl.rcParams['axes.labelcolor'] = 'black'
mpl.rcParams['figure.facecolor'] = 'white'
mpl.rcParams['axes.facecolor'] = 'white'
mpl.rcParams['axes.grid'] = False

def apply_journal_style(ax):
    for label in ax.get_xticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')
        label.set_color('black')
    for label in ax.get_yticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')
        label.set_color('black')
    ax.xaxis.label.set_fontweight('bold')
    ax.yaxis.label.set_fontweight('bold')
    ax.title.set_fontweight('bold')
    for spine in ax.spines.values():
        spine.set_visible(True)
        spine.set_color('black')
        spine.set_linewidth(2.0)
    ax.grid(False)

# =====
# REGRESSION PLOT (shows Train R^2, Test R^2 and Pearson R on the plot itself)
#
# Pearson R and the red trend line are computed on the TEST SET ONLY, so the
# number printed on this figure is identical to "Pearson R" in the final
# comparison table.
# =====
```

```

def regression_plot(y_train, y_train_pred, y_test, y_test_pred, model_name):

    plt.rcParams['font.family'] = 'Times New Roman'

    fig, ax = plt.subplots(figsize=(7, 7), dpi=600)
    ax.set_facecolor('white')
    ax.grid(False)

    all_actual = np.concatenate([y_train, y_test])
    all_pred = np.concatenate([y_train_pred, y_test_pred])

    ax.scatter(y_train, y_train_pred, c="#8E44AD", s=55, alpha=0.75,
               edgecolors="black", linewidths=0.4, label="Train")
    ax.scatter(y_test, y_test_pred, c="#2980B9", s=55, alpha=0.80,
               edgecolors="black", linewidths=0.4, label="Test")

    mn = min(all_actual.min(), all_pred.min())
    mx = max(all_actual.max(), all_pred.max())
    padding = (mx - mn) * 0.05
    mx += padding
    mn = 0.0

    ax.plot([mn, mx], [mn, mx], '--', color='dimgray', linewidth=1.5)

    # Trend Line + Pearson R fit on the TEST SET only (consistent with the table)
    slope, intercept, r_value, p_value, std_err = linregress(y_test, y_test_pred)
    x_line = np.linspace(mn, mx, 100)
    y_line = intercept + slope * x_line
    ax.plot(x_line, y_line, color='red', linewidth=2)

    # ---- Metrics: Train R^2, Test R^2, Pearson R (test set) ----
    r2_train = r2_score(y_train, y_train_pred)
    r2_test = r2_score(y_test, y_test_pred)
    r_pearson = np.corrcoef(y_test, y_test_pred)[0, 1]

    metrics_text = (
        f"Train R\u00b2 = {r2_train:.3f}\n"
        f"Test R\u00b2 = {r2_test:.3f}\n"
        f"R (test) = {r_pearson:.3f}\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )

    ax.text(
        0.04, 0.96, metrics_text, transform=ax.transAxes, fontsize=11,
        fontweight='bold', fontfamily='Times New Roman', va='top',
        bbox=dict(facecolor='white', edgecolor='black', linewidth=2.5,
                  alpha=1.0, boxstyle='round,pad=0.5')
    )

    ax.set_xlabel("Observed Values \uA7D8/r", fontsize=15, fontweight='bold', fo
    ax.set_ylabel("Predicted Values \uA7D8/r", fontsize=15, fontweight='bold', f
    ax.set_title(model_name, fontsize=16, fontweight='bold', fontfamily='Times N

    ax.set_xlim(0.0, mx)
    ax.set_ylim(0.0, mx)

    # Adaptive tick spacing: aim for a fine grid without ever generating
    # an unreasonable number of ticks if the target's range is large.
    tick_interval = 0.05
    n_ticks = (mx - mn) / tick_interval

```

```

if n_ticks > 25:
    tick_interval = (mx - mn) / 10

x_ticks = np.arange(0.0, mx + tick_interval, tick_interval)
ax.set_xticks(x_ticks)
ax.set_yticks(x_ticks)

ax.tick_params(axis='both', labelsz=13)
for label in ax.get_xticklabels():
    label.set_fontweight('bold')
    label.set_fontfamily('Times New Roman')
for label in ax.get_yticklabels():
    label.set_fontweight('bold')
    label.set_fontfamily('Times New Roman')

ax.set_aspect('equal')

for spine in ax.spines.values():
    spine.set_visible(True)
    spine.set_color('black')
    spine.set_linewidth(2.5)

legend = ax.legend(fontsize=11, frameon=True, loc='lower right')
for text in legend.get_texts():
    text.set_fontweight('bold')
    text.set_fontfamily('Times New Roman')

apply_journal_style(ax)
plt.tight_layout()

plt.savefig(os.path.join(OUTPUT_DIR, f"{model_name}_Regression.png"), dpi=60)
plt.show()
plt.close()

return r2_train, r2_test, r_pearson

# =====
# FEATURE IMPORTANCE PLOT
# =====
def feature_importance_plot(model, feature_names, model_name, top_n=None):
    """
    Plots feature importance as PERCENTAGES (normalized to sum to 100).
    If top_n is set, only the top_n features are shown -> the plotted bars
    will NOT sum to 100 in that case (this is expected, not a bug).
    """
    raw_importance = np.asarray(model.feature_importances_, dtype=float)

    # Normalize once, here, so plot and printed values always agree
    importance_pct = raw_importance / raw_importance.sum() * 100

    importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance (%)": importance_pct
    }).sort_values("Importance (%)", ascending=True)

    full_sum = importance_df["Importance (%)"].sum() # sanity check, should be

    plot_df = importance_df.copy()
    if top_n:

```

```

        plot_df = plot_df.tail(top_n)
        shown_sum = plot_df["Importance (%)"].sum()
        print(f"NOTE: showing top {top_n} of {len(importance_df)} features "
              f"({shown_sum:.2f}% of total importance shown; full set = {full_su

plt.style.use('default')
plt.rcParams['font.family'] = 'Times New Roman'

fig, ax = plt.subplots(figsize=(5, 3), dpi=600)
ax.grid(False)

sns.barplot(data=plot_df, x="Importance (%)", y="Feature", palette="viridis")

for i, v in enumerate(plot_df["Importance (%)"]):
    ax.text(v, i, f" {v:.1f}%", va='center', fontsize=8)

ax.set_xlabel("Feature Importance (%)")
ax.set_ylabel("")
ax.set_title(f"{model_name} - Feature Importance")

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, f"{model_name}_FeatureImportance.png"),
            dpi=600, bbox_inches='tight')
plt.show()
plt.close()

return importance_df # full (untruncated) table, for reuse / printing / san

# =====
# SHAP PLOTS
# =====
def shap_beeswarm_plot(shap_values, X, model_name, max_display=15):
    plt.rcParams['font.family'] = 'Times New Roman'

    shap.summary_plot(shap_values, X, max_display=max_display, show=False)
    ax = plt.gca()

    for label in ax.get_xticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')
    for label in ax.get_yticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')

    ax.xaxis.label.set_fontweight('bold')
    ax.yaxis.label.set_fontweight('bold')
    ax.xaxis.label.set_fontfamily('Times New Roman')
    ax.yaxis.label.set_fontfamily('Times New Roman')

    for spine in ax.spines.values():
        spine.set_linewidth(1.5)

    plt.title(f"{model_name} SHAP Beeswarm", fontsize=16, fontweight='bold', for
    plt.tight_layout()

    plt.savefig(os.path.join(OUTPUT_DIR, f"{model_name}_SHAP_Beeswarm.png"), dpi
    plt.show()
    plt.close()

```

```

def shap_bar_plot(shap_values, X, model_name, max_display=15):
    plt.rcParams['font.family'] = 'Times New Roman'

    shap.summary_plot(shap_values, X, plot_type="bar", max_display=max_display,
ax = plt.gca())

    for label in ax.get_xticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')
    for label in ax.get_yticklabels():
        label.set_fontweight('bold')
        label.set_fontfamily('Times New Roman')

    ax.xaxis.label.set_fontweight('bold')
    ax.yaxis.label.set_fontweight('bold')
    ax.xaxis.label.set_fontfamily('Times New Roman')
    ax.yaxis.label.set_fontfamily('Times New Roman')

    for spine in ax.spines.values():
        spine.set_linewidth(1.5)

    plt.title(f"{model_name} SHAP Feature Importance", fontsize=16, fontweight='bold')
    plt.tight_layout()

    plt.savefig(os.path.join(OUTPUT_DIR, f"{model_name}_SHAP_Bar.png"), dpi=600,
    plt.show()
    plt.close()

```

5. Models & hyperparameter grids (GridSearchCV)

```

In [ ]: cv = KFold(n_splits=5, shuffle=True, random_state=42)

base_models = {
    "Random Forest": RandomForestRegressor(random_state=42, n_jobs=-1, bootstrap
    "LightGBM": LGBMRegressor(random_state=42, verbose=-1),
    "XGBoost": XGBRegressor(random_state=42, verbosity=0, enable_categorical=False
    "CatBoost": CatBoostRegressor(random_seed=42, loss_function="RMSE", verbose=
}

param_grids = {
    "Random Forest": {
        "n_estimators": [100, 200, 300, 400, 500],
        "max_depth": [5, 10, 15, 20, 25],
        "min_samples_leaf": [1, 2, 3, 4, 5],
        "max_features": ["sqrt", "log2", None, 0.5, 0.7],
    },
    "LightGBM": {
        "n_estimators": [100, 200, 300, 400, 500],
        "learning_rate": [0.01, 0.03, 0.05, 0.07, 0.1],
        "num_leaves": [15, 31, 47, 63, 95],
        "max_depth": [-1, 5, 10, 15, 20],
    },
    "XGBoost": {
        "n_estimators": [100, 200, 300, 400, 500],
        "learning_rate": [0.01, 0.03, 0.05, 0.07, 0.1],
        "max_depth": [3, 4, 6, 8, 10],
    },
}

```

```

    "CatBoost": {
        "iterations": [100, 200, 300, 400, 500],
        "learning_rate": [0.01, 0.03, 0.05, 0.07, 0.1],
        "depth": [4, 6, 8, 10, 12],
    },
}

feature_names = list(X.columns)
target = 'S/r'
print("Features:", feature_names)
for name, grid in param_grids.items():
    n_combos = 1
    for v in grid.values():
        n_combos *= len(v)
    print(f"{name}: {n_combos} parameter combinations x {cv.get_n_splits()} fold

```

6. Grid-search tuning, training, evaluation & plots

```

In [ ]: results = []
best_params_all = {}

for name, base_model in base_models.items():

    print(f"\n===== {name} =====")

    # ---- Hyperparameter tuning (GridSearchCV, 5-fold CV on the TRAINING set) ----
    grid_search = GridSearchCV(
        estimator=base_model,
        param_grid=param_grids[name],
        cv=cv,
        scoring="r2",
        n_jobs=-1,
        refit=True,
    )
    grid_search.fit(X_train, y_train)

    model = grid_search.best_estimator_
    print(f"Best CV R\u00b2 : {grid_search.best_score_:.4f}")
    print(f"Best Params : {grid_search.best_params_}")
    best_params_all[name] = grid_search.best_params_

    # Predict with the tuned model
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # ---- Metrics (all test-set-based, except Train R^2) ----
    train_r2 = r2_score(y_train, y_train_pred)
    test_r2 = r2_score(y_test, y_test_pred)
    r_pearson = np.corrcoef(y_test, y_test_pred)[0, 1]

    mse = mean_squared_error(y_test, y_test_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_test_pred)
    mape = mean_absolute_percentage_error(y_test, y_test_pred)
    bias = np.mean(y_test_pred - y_test)

    print(f"Train R\u00b2 : {train_r2:.4f}")
    print(f"Test R\u00b2 : {test_r2:.4f}")

```

```

print(f"Pearson R : {r_pearson:.4f}")
print(f"MSE      : {mse:.6f}")
print(f"RMSE     : {rmse:.6f}")
print(f"MAE      : {mae:.6f}")
print(f"MAPE     : {mape:.4f}")

results.append([
    name, train_r2, test_r2, r_pearson, mse, rmse, mae, mape, bias,
    str(grid_search.best_params_)
])

# Regression plot (also annotated with Train R^2 / Test R^2 / R, all consist
regression_plot(y_train, y_train_pred, y_test, y_test_pred, model_name=name)

# Feature importance (tree-based models expose feature_importances_)
if hasattr(model, "feature_importances_"):
    importance_df = feature_importance_plot(model, feature_names, model_name

    print("\nFeature Importance (%):")
    print(importance_df.sort_values("Importance (%)", ascending=False)
          .set_index("Feature")["Importance (%)"]
          .round(2))

    unrounded_sum = importance_df["Importance (%)"].sum()
    rounded_sum = importance_df["Importance (%)"].round(2).sum()
    print(f"Sum of unrounded percentages: {unrounded_sum:.6f}")
    print(f"Sum of rounded percentages: {rounded_sum:.2f}")

# SHAP analysis
print("\nSHAP Analysis...")
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)

shap_beeswarm_plot(shap_values, X_test, model_name=name, max_display=len(fea
shap_bar_plot(shap_values, X_test, model_name=name, max_display=len(feature_

shap_importance = np.abs(shap_values.values).mean(axis=0)
shap_importance = pd.Series(shap_importance, index=feature_names).sort_value
print("\nSHAP Feature Ranking:")
print(shap_importance)

print("\n==== Done tuning & training all models =====")

```

7. Final comparison table

```

In [ ]: results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Train R²",
        "Test R²",
        "Pearson R",
        "MSE",
        "RMSE",
        "MAE",
        "MAPE",
        "Bias",
        "Best Params (GridSearchCV)"
    ]
)

```

```

    ]
)

results_df

```

```

In [ ]: key_cols = ["Train R2", "Test R2", "Pearson R"]

def highlight_key_cols(col):
    return ['background-color: #FFF2AC; font-weight: bold' if col.name in key_co

def sig4(v):
    """Format a number to 4 significant figures as a string."""
    return f"{v:.4g}"

display_df = results_df.copy()
numeric_cols = display_df.select_dtypes(include="number").columns

styled_table = (
    display_df.style
    .apply(highlight_key_cols, axis=0)
    .format({c: sig4 for c in numeric_cols})
    .set_caption("Model Comparison (GridSearchCV-tuned)")
)
styled_table

```

```

In [ ]: table_values = display_df.drop(columns=["Best Params (GridSearchCV)"]).copy()
for c in numeric_cols:
    table_values[c] = table_values[c].map(sig4)

fig, ax = plt.subplots(figsize=(14, 5))
ax.axis("off")

table = ax.table(
    cellText=table_values.values,
    colLabels=table_values.columns,
    cellLoc="center",
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.8)

key_col_idx = [table_values.columns.get_loc(c) for c in key_cols]

for (row, col), cell in table.get_celld().items():
    cell.set_linewidth(0.8)
    if row == 0:
        cell.set_text_props(weight="bold")
    if col in key_col_idx and row > 0:
        cell.set_facecolor("#FFF2AC")
    if col in key_col_idx and row == 0:
        cell.set_facecolor("#FFE066")

plt.title("Model Comparison – Train R2, Test R2, Pearson R", fontsize=14, fontwe
plt.tight_layout()

plt.savefig(
    os.path.join(OUTPUT_DIR, "Model_Comparison_Table.png"),

```

```

    dpi=600,
    bbox_inches="tight"
)

plt.show()

```

```

In [ ]: for name, params in best_params_all.items():
        print(f"{name}:")
        for k, v in params.items():
            print(f"    {k}: {v}")
        print()

```

8.1 Settings, re-fit of the tuned models, tidy test-only error table Nothing is re-tuned: the models are re-fitted with best_params_all and the cell checks that the test R^2 of Section 7 is reproduced.

```

In [ ]: # =====
# 8.1 SETTINGS, RE-FIT OF THE TUNED MODELS, AND ONE TIDY TEST-ONLY ERROR TABLE
# =====

from sklearn.base import clone
from sklearn.neighbors import NearestNeighbors
from scipy import stats

R12_DIR = os.path.join(OUTPUT_DIR, "Reviewer12_ErrorAnalysis")
os.makedirs(R12_DIR, exist_ok=True)

# ----- general settings -----
PLOT_MODELS = list(base_models.keys()) # e.g. ["CatBoost"] to plot only one
REGIME_LABELS = ["Thin", "Intermediate", "Thick"]
REGIME_EDGES = None # None -> terciles of the full target. Or give physical
                    # in target units, e.g. [0.05, 0.15] (len = len(REGIME_LABELS))
BIN_VARS = list(feature_cols) # tables: error vs range for ALL six
FIG_VARS_MAIN = ["Ca", "Re", "We"] # main-text figure R12-3 (what the reader
                                   # the remaining inputs go to a supplementary figure)
MAP_PAIRS = [("Ca", "Re"), ("Ca", "We")] # variable pairs for the 2-D error
N_BINS = 5 # equal-count bins per variable
OUTLIER_BASIS = "resid" # "resid" (absolute residual) or "rel_err" (relative error)
OUTLIER_WITHIN_REGIME = True # True: score each point against the scatter of IT
                             # (A single global threshold on absolute residuals would
                             # every thick-film point simply because the error grows)
OUTLIER_Z = 3.0 # flag test points with |robust z| > this value
K_NEIGHBOURS = 5 # neighbours used for the local training-density measurement
N_BOOT = 2000 # bootstrap resamples for confidence intervals
FIG_SAVE_DPI = 600

# ----- EDIT THESE -----
# Capillary radii (mm) for which the error is converted to a dimensional film
# thickness. The values below are PLACEHOLDERS -> replace by the radii of your
# experiments.
CAPILLARY_RADII_MM = [0.25, 0.5, 1.0]
# Experimental uncertainty of the measured film thickness. Leave both as None
# until you fill them in; 8.8 then skips the comparison instead of inventing one
EXP_ABS_UNC_UM = None # e.g. 2.0 -> +/- 2 micrometre (optical resolution, current)
EXP_REL_UNC = None # e.g. 0.05 -> +/- 5 % of the measured thickness
# If both are given they are combined in quadrature: U = sqrt(U_abs^2 + (U_rel*h
# -----

assert (y > 0).all(), "target must be strictly positive for relative errors"

```

```

TL = target                                     # column label of the target, e.g.

# ---- 1) Re-fit each model with its GridSearchCV-selected hyper-parameters ----
#      (no new search; same random_state -> same model as in Section 6)
fitted = {}
for name in base_models:
    m = clone(base_models[name]).set_params(**best_params_all[name])
    m.fit(X_train, y_train)
    fitted[name] = m

# sanity check against the table of Section 7 (test R^2 must be reproduced)
for name, m in fitted.items():
    r2_now = r2_score(y_test, m.predict(X_test))
    r2_tab = results_df.set_index("Model").loc[name, "Test R^2"]
    flag = "OK " if abs(r2_now - r2_tab) < 1e-4 else "DIFFERS!"
    print(f"{flag} {name:14s} test R2 re-fit = {r2_now:.4f} | Section 7 table =

# ---- 2) Regime definition (thin / intermediate / thick) -----
if REGIME_EDGES is None:
    REGIME_EDGES = list(np.quantile(y.values, np.linspace(0, 1, len(REGIME_LABEL
assert len(REGIME_EDGES) == len(REGIME_LABELS) - 1
_reg_bins = [-np.inf] + list(REGIME_EDGES) + [np.inf]

def regime_of(v):
    return pd.cut(np.asarray(v), bins=_reg_bins, labels=REGIME_LABELS)

def regime_range_text(i):
    if i == 0:
        return f"< {REGIME_EDGES[0]:.3g}"
    if i == len(REGIME_LABELS) - 1:
        return f"> {REGIME_EDGES[-1]:.3g}"
    return f"{REGIME_EDGES[i-1]:.3g}-{REGIME_EDGES[i]:.3g}"

reg_palette = ["#2E86C1", "#F39C12", "#C0392B", "#7D3C98", "#27AE60"]
reg_col = dict(zip(REGIME_LABELS, reg_palette))

# ---- 3) Equal-count bins for Ca / Re / We (edges from the FULL data set) ----
bin_edges = {v: np.unique(np.quantile(data[v].values, np.linspace(0, 1, N_BINS +
    for v in BIN_VARS}

def bin_labels(e):
    return [f"{e[i]:.3g}-{e[i+1]:.3g}" for i in range(len(e) - 1)]

def assign_bin(vals, v):
    return pd.cut(np.asarray(vals), bins=bin_edges[v], labels=bin_labels(bin_edg
        include_lowest=True)

# ---- 4) Local training density (mean distance to K nearest TRAINING points) --
#      in the standardised feature space that the models actually see
nn = NearestNeighbors(n_neighbors=K_NEIGHBOURS).fit(X_train.values)
_dist, _nbr = nn.kneighbors(X_test.values)
knn_dist = _dist.mean(axis=1)
nbr_y_min = y_train.values[_nbr].min(axis=1)
nbr_y_max = y_train.values[_nbr].max(axis=1)

# ---- 5) Tidy table: one row per (model, test point) -----
def robust_z(v):
    med = np.median(v)
    mad = np.median(np.abs(v - med))
    return (v - med) / (1.4826 * mad) if mad > 0 else np.zeros_like(v, dtype=flo

```

```

feat_raw_test = data.loc[X_test.index, feature_cols] # un-scaled feature
frames = []
for name in base_models:
    p = fitted[name].predict(X_test)
    d = feat_raw_test.copy()
    d.insert(0, "row_id", X_test.index.values)
    d.insert(0, "Model", name)
    d["y_true"] = y_test.values
    d["y_pred"] = p
    d["resid"] = p - y_test.values # predicted - obser
    d["rel_err"] = d["resid"] / d["y_true"]
    d["abs_err"] = d["resid"].abs()
    d["abs_rel_err"] = d["rel_err"].abs()
    d["Regime"] = regime_of(d["y_true"])
    basis = d["resid"] if OUTLIER_BASIS == "resid" else d["rel_err"]
    if OUTLIER_WITHIN_REGIME:
        d["z_robust"] = basis.groupby(d["Regime"], observed=True).transform(
            lambda v: pd.Series(robust_z(v.values), index=v.index))
    else:
        d["z_robust"] = robust_z(basis.values)
    d["is_outlier"] = d["z_robust"].abs() > OUTLIER_Z
    for v in BIN_VARS:
        d[f"bin_{v}"] = assign_bin(d[v], v)
    d["knn_dist"] = knn_dist
    d["nbr_y_min"] = nbr_y_min
    d["nbr_y_max"] = nbr_y_max
    frames.append(d)
err_all = pd.concat(frames, ignore_index=True)

# training-set counts per regime / bin (used to show how well each range is samp
train_regime = pd.Series(regime_of(y_train.values)).value_counts()
train_bin_counts = {v: pd.Series(assign_bin(data.loc[X_train.index, v], v)).valu
    for v in BIN_VARS}

print(f"\nTest points per model: {len(y_test)} | regimes (target units): "
      + ", ".join(f"{l} [{regime_range_text(i)}]" for i, l in enumerate(REGIME_L
print("Flagged outliers per model (|robust z| > %.1f, basis = %s, %s):"
      % (OUTLIER_Z, OUTLIER_BASIS, "within regime" if OUTLIER_WITHIN_REGIME else
print(err_all.groupby("Model")["is_outlier"].sum().to_string())

```

8.2 Helpers and Table A — overall test-only metrics and heteroscedasticity check

```

In [ ]: # =====
# 8.2 HELPERS + TABLE A: OVERALL TEST-ONLY METRICS (incl. heteroscedasticity)
# =====
_rng = np.random.default_rng(42)

def r12_style():
    """Re-assert the journal style (plt.style.use('default') inside the earlier
    feature-importance function resets rcParams)."""
    mpl.rcParams.update({
        'font.family': 'Times New Roman', 'font.weight': 'bold',
        'axes.labelweight': 'bold', 'axes.titleweight': 'bold',
        'axes.edgecolor': 'black', 'axes.linewidth': 2.0,
        'xtick.major.width': 2.0, 'ytick.major.width': 2.0,
        'figure.facecolor': 'white', 'axes.facecolor': 'white', 'axes.grid': Fal

```

```

    })

def boot_ci(v, stat=np.mean, n=N_BOOT, alpha=0.05):
    """Percentile bootstrap CI of a statistic (needs >= 3 values)."""
    v = np.asarray(v, dtype=float)
    if len(v) < 3:
        return (np.nan, np.nan)
    idx = _rng.integers(0, len(v), size=(n, len(v)))
    return tuple(np.quantile(stat(v[idx], axis=1), [alpha / 2, 1 - alpha / 2]))

def running_stats(x, yv, nb=8):
    """Equal-count running mean and std of yv along x (no extra dependencies)."""
    x, yv = np.asarray(x), np.asarray(yv)
    q = np.unique(np.quantile(x, np.linspace(0, 1, nb + 1)))
    k = np.clip(np.digitize(x, q[1:-1]), 0, len(q) - 2)
    xc, m, s = [], [], []
    for b in range(len(q) - 1):
        sel = k == b
        if sel.sum() >= 2:
            xc.append(np.median(x[sel])); m.append(yv[sel].mean()); s.append(yv[
    return np.array(xc), np.array(m), np.array(s)

def summarize(df):
    """Error statistics of one subset of test points (target units and %)."""
    r, rel = df["resid"].values, df["rel_err"].values
    return pd.Series({
        "n": len(df),
        "Bias (pred-obs)": r.mean(),
        "MAE": np.abs(r).mean(),
        "RMSE": np.sqrt((r ** 2).mean()),
        "P95 |err|": np.quantile(np.abs(r), 0.95),
        "MAPE (%)": 100 * np.abs(rel).mean(),
        "Mean rel. err (%)": 100 * rel.mean(),
        "P95 |rel. err| (%)": 100 * np.quantile(np.abs(rel), 0.95),
        "Max |rel. err| (%)": 100 * np.abs(rel).max(),
    })

def savefig_r12(fig, fname):
    fig.savefig(os.path.join(R12_DIR, fname), dpi=FIG_SAVE_DPI, bbox_inches="tight",
        facecolor="white")

def panel_tag(ax, tag):
    ax.text(-0.13, 1.04, tag, transform=ax.transAxes, fontsize=15, fontweight="bold")

# ----- TABLE A: overall -----
rows = []
for name in base_models:
    d = err_all[err_all.Model == name]
    s = summarize(d)
    rho_a, p_a = stats.spearmanr(d["y_true"], d["abs_err"]) # abs. error g
    rho_r, p_r = stats.spearmanr(d["y_true"], d["abs_rel_err"]) # rel. error g
    s["Test R²"] = r2_score(d["y_true"], d["y_pred"])
    s["Spearman ρ(|err|, obs)"] = rho_a
    s["p-value"] = p_a
    s["Spearman ρ(|rel err|, obs)"] = rho_r
    s["p-value "] = p_r
    s.name = name
    rows.append(s)
table_A = pd.DataFrame(rows)
table_A.index.name = "Model"

```

```
cols_first = ["n", "Test R2", "Bias (pred-obs)", "MAE", "RMSE", "P95 |err|", "MAE",
              "Mean rel. err (%)", "P95 |rel. err| (%)", "Max |rel. err| (%)"]
table_A = table_A[cols_first + [c for c in table_A.columns if c not in cols_first]]
print("TABLE A - test-only error statistics (target units unless stated)")
display(table_A.style.format(precision=4))
print("\nSpearman  $\rho(|err|, obs) > 0$  with small  $p \Rightarrow$  absolute error grows with fi
      "(heteroscedastic);  $\rho(|rel\ err|, obs)$  shows whether the RELATIVE error als
```

8.3 Figure R12-1 — test-only parity and residual diagnostics

Parity (test only, $\pm 10\%$ / $\pm 20\%$ bands), residual vs observed / predicted, relative error vs observed, residual histogram and Q-Q plot. Colours = thin / intermediate / thick regime.

```
In [ ]: # =====
# 8.3 FIGURE R12-1: TEST-ONLY PARITY + RESIDUAL DIAGNOSTICS (one figure / mode
# =====
def fig_test_only_residuals(name):
    r12_style()
    d = err_all[err_all.Model == name]
    yt, yp, res, rel = d.y_true.values, d.y_pred.values, d.resid.values, d.rel_e
    fig, axes = plt.subplots(2, 3, figsize=(17, 10.5))

    # (a) parity, TEST ONLY, coloured by regime, +/-10 % and +/-20 % bands
    ax = axes[0, 0]
    for lab in REGIME_LABELS:
        s = d[d.Regime == lab]
        ax.scatter(s.y_true, s.y_pred, s=45, c=reg_col[lab], edgecolors="black",
                  linewidths=0.4, alpha=0.85, label=lab)
    mx = max(yt.max(), yp.max()) * 1.05
    xx = np.linspace(0, mx, 50)
    ax.plot(xx, xx, "--", color="dimgray", lw=1.5)
    ax.plot(xx, xx * 1.1, ":", color="gray", lw=1.2, label="±10 %")
    ax.plot(xx, xx * 0.9, ":", color="gray", lw=1.2)
    ax.plot(xx, xx * 1.2, "-.", color="gray", lw=1.0, label="±20 %")
    ax.plot(xx, xx * 0.8, "-.", color="gray", lw=1.0)
    ax.set_xlim(0, mx); ax.set_ylim(0, mx); ax.set_aspect("equal")
    ax.set_xlabel(f"Observed {TL}", fontsize=13); ax.set_ylabel(f"Predicted {TL}")
    txt = (f"Test R2 = {r2_score(yt, yp):.3f}\nMAPE = {100*np.abs(rel).mean():.1f}
          f"Bias = {res.mean():+.4f}\nn(test) = {len(d)}")
    ax.text(0.04, 0.96, txt, transform=ax.transAxes, fontsize=10.5, va="top", fc
           bbox=dict(facecolor="white", edgecolor="black", linewidth=1.5, boxst
    ax.legend(fontsize=9, loc="lower right", frameon=True)
    ax.set_title("Test-only parity", fontsize=13)

    # (b) residual vs observed
    ax = axes[0, 1]
    for lab in REGIME_LABELS:
        s = d[d.Regime == lab]
        ax.scatter(s.y_true, s.resid, s=40, c=reg_col[lab], edgecolors="black",
    xc, m, sd = running_stats(yt, res)
    ax.axhline(0, color="k", lw=1.3)
    ax.plot(xc, m, "r-o", ms=4, lw=2, label="running mean ± 1 SD")
    ax.fill_between(xc, m - sd, m + sd, color="red", alpha=0.15)
    rho, p = stats.spearmanr(yt, np.abs(res))
    ax.text(0.04, 0.96, f"Spearman  $\rho(|res|, obs) = \{rho:.2f\}\backslash np = \{p:.2g\}$ ", tran
           fontsize=10, va="top", fontweight="bold",
           bbox=dict(facecolor="white", edgecolor="black", linewidth=1.2, boxst
    ax.set_xlabel(f"Observed {TL}", fontsize=13); ax.set_ylabel("Residual (pred
```

```

ax.legend(fontsize=9, loc="lower left"); ax.set_title("Residual vs observed")

# (c) residual vs predicted
ax = axes[0, 2]
for lab in REGIME_LABELS:
    s = d[d.Regime == lab]
    ax.scatter(s.y_pred, s.resid, s=40, c=reg_col[lab], edgecolors="black",
               xc, m, sd = running_stats(y, res)
    ax.axhline(0, color="k", lw=1.3)
    ax.plot(xc, m, "r-o", ms=4, lw=2); ax.fill_between(xc, m - sd, m + sd, color
    ax.set_xlabel(f"Predicted {TL}", fontsize=13); ax.set_ylabel("Residual (pred
    ax.set_title("Residual vs predicted", fontsize=13)

# (d) relative error vs observed
ax = axes[1, 0]
for lab in REGIME_LABELS:
    s = d[d.Regime == lab]
    ax.scatter(s.y_true, 100 * s.rel_err, s=40, c=reg_col[lab], edgecolors="
    xc, m, sd = running_stats(yt, 100 * rel)
    ax.axhline(0, color="k", lw=1.3)
    for lv in (-10, 10):
        ax.axhline(lv, color="gray", ls=":", lw=1.2)
    ax.plot(xc, m, "r-o", ms=4, lw=2); ax.fill_between(xc, m - sd, m + sd, color
    ax.set_xlabel(f"Observed {TL}", fontsize=13); ax.set_ylabel("Relative error
    ax.set_title("Relative error vs observed", fontsize=13)

# (e) residual histogram + normal fit
ax = axes[1, 1]
ax.hist(res, bins=20, density=True, color="#2980B9", edgecolor="black", alph
mu, sg = res.mean(), res.std(ddof=1)
xg = np.linspace(res.min() - sg, res.max() + sg, 200)
ax.plot(xg, stats.norm.pdf(xg, mu, sg), "r-", lw=2, label="Normal fit")
ax.axvline(0, color="k", lw=1.3)
ax.text(0.04, 0.96, f"mean = {mu:+.4f}\nSD = {sg:.4f}\nskew = {stats.skew(re
        f"ex. kurtosis = {stats.kurtosis(res):.2f}", transform=ax.transAxes,
        fontweight="bold", bbox=dict(facecolor="white", edgecolor="black", l
    ax.set_xlabel("Residual (pred - obs)", fontsize=13); ax.set_ylabel("Probabil
    ax.legend(fontsize=9, loc="upper right"); ax.set_title("Residual distributio

# (f) normal Q-Q
ax = axes[1, 2]
(osm, osr), (sl, ic, _) = stats.probplot(res, dist="norm")
ax.scatter(osm, osr, s=35, c="#2980B9", edgecolors="black", linewidths=0.4)
ax.plot(osm, sl * osm + ic, "r-", lw=2)
ax.set_xlabel("Theoretical quantiles", fontsize=13); ax.set_ylabel("Ordered
ax.set_title("Normal Q-Q plot", fontsize=13)

for ax, tg in zip(axes.ravel(), "abcdef"):
    ax.tick_params(axis="both", labelsize=11)
    apply_journal_style(ax)
    panel_tag(ax, f"({tg})")
fig.suptitle(f"{name} - test-only diagnostics (n = {len(d)})", fontsize=16,
fig.tight_layout()
savefig_r12(fig, f"{name}_R12_1_TestOnly_Residuals.png")
plt.show(); plt.close(fig)

for _n in PLOT_MODELS:
    fig_test_only_residuals(_n)

```

8.4 Figure R12-2 and Table B — thin / intermediate / thick film regimes

By default the regimes are terciles of the target; set `REGIME_EDGES` in 8.1 for physically motivated cut-offs.

```
In [ ]: # =====
# 8.4 THIN / INTERMEDIATE / THICK FILM ERRORS (TABLE B + FIGURE R12-2)
# =====
rows = []
for name in base_models:
    for i, lab in enumerate(REGIME_LABELS):
        s = err_all[(err_all.Model == name) & (err_all.Regime == lab)]
        if len(s) == 0:
            continue
        r = summarize(s)
        lo, hi = boot_ci(s["abs_rel_err"].values * 100)
        r["MAPE 95% CI low"], r["MAPE 95% CI high"] = lo, hi
        r["Model"], r["Regime"], r["Range (target)"] = name, lab, regime_range_t
        r["n(train)"] = int(train_regime.get(lab, 0))
        rows.append(r)
table_B = pd.DataFrame(rows).set_index(["Model", "Regime"])
front = ["Range (target)", "n", "n(train)"]
table_B = table_B[front + [c for c in table_B.columns if c not in front]]
print("TABLE B - test-only error by film-thickness regime")
display(table_B.style.format(precision=4))

def fig_regime(name):
    r12_style()
    d = err_all[err_all.Model == name]
    labs = [l for l in REGIME_LABELS if (d.Regime == l).any()]
    tick = [f"{l}\n{regime_range_text(REGIME_LABELS.index(l))}\n(n={int((d.Regime == l).any())})" for l in labs]
    fig, axes = plt.subplots(1, 3, figsize=(17, 5.6))

    def boxes(ax, values, ylabel, title, zero_line=True):
        bp = ax.boxplot(values, positions=range(len(labs)), widths=0.55, patch_artist=True, showfliers=False, medianprops=dict(color="red", lw=2.2))
        for patch, l in zip(bp["boxes"], labs):
            patch.set_facecolor(reg_col[l]); patch.set_alpha(0.45); patch.set_linewidth(1.5)
        for k, (v, l) in enumerate(zip(values, labs)):
            ax.scatter(k + _rng.normal(0, 0.07, len(v)), v, s=22, c=reg_col[l], edgecolors="black", linewidths=0.4, alpha=0.85, zorder=3)
        if zero_line:
            ax.axhline(0, color="k", lw=1.2)
        ax.set_xticks(range(len(labs))); ax.set_xticklabels(tick)
        ax.set_ylabel(ylabel, fontsize=13); ax.set_title(title, fontsize=13)

    boxes(axes[0], [100 * d[d.Regime == l].rel_err.values for l in labs],
          "Relative error (%)", "Signed relative error")
    boxes(axes[1], [d[d.Regime == l].abs_err.values for l in labs],
          f"|Error| in {TL} units", "Absolute error", zero_line=False)

    ax = axes[2]
    # MAPE with bootstrap
    mape = [100 * d[d.Regime == l].abs_rel_err.mean() for l in labs]
    ci = [boot_ci(100 * d[d.Regime == l].abs_rel_err.values) for l in labs]
    err = np.array([(m - c[0], c[1] - m) for m, c in zip(mape, ci)]).T
    ax.bar(range(len(labs)), mape, color=[reg_col[l] for l in labs], edgecolor="black",
           linewidth=1.5, alpha=0.75, yerr=np.nan_to_num(err), capsize=6, error_kw={})
```

```

for k, l in enumerate(labs):
    ax.text(k, 0.02 * max(mape), f"n_train = {int(train_regime.get(l, 0))}",
           va="bottom", fontsize=10, fontweight="bold", rotation=90 if len(
ax.set_xticks(range(len(labs))); ax.set_xticklabels(tick)
ax.set_ylabel("MAPE (%) with 95 % bootstrap CI", fontsize=13); ax.set_title(

for ax, tg in zip(axes, "abc"):
    ax.tick_params(axis="both", labelsize=11); apply_journal_style(ax); pane
fig.suptitle(f"{name} - test-only error by film-thickness regime", fontsize=
fig.tight_layout()
savefig_r12(fig, f"{name}_R12_2_Regime_Errors.png")
plt.show(); plt.close(fig)

for _n in PLOT_MODELS:
    fig_regime(_n)

```

8.5 Figures R12-3 / R12-3b and Table C — error across the range of every input

Equal-count bins from the full data set; labels show `n_test / n_train` per bin; error bars are 95 % bootstrap CIs. **R12-3** (Ca, Re, We) is the main-text figure the reviewer asked for; **R12-3b** (Bo, V.R, D.R) is for the supplement. Bo, V.R and D.R are constant within an experimental series and take few distinct values, so their bins can merge.

```

In [ ]: # =====
# 8.5 ERROR vs RANGE OF EACH INPUT (TABLE C + FIGURES R12-3 / R12-3b)
# =====

rows = []
for name in base_models:
    for v in BIN_VARS:
        labs = bin_labels(bin_edges[v])
        for lab in labs:
            s = err_all[(err_all.Model == name) & (err_all[f"bin_{v}"] == lab)]
            if len(s) == 0:
                continue
            r = summarize(s)
            lo, hi = boot_ci(s["abs_err"].values)
            r["MAE 95% CI low"], r["MAE 95% CI high"] = lo, hi
            r["Model"], r["Variable"], r["Bin"] = name, v, lab
            r["n(train)"] = int(train_bin_counts[v].get(lab, 0))
            rows.append(r)
table_C = pd.DataFrame(rows).set_index(["Model", "Variable", "Bin"])
front = ["n", "n(train)"]
table_C = table_C[front + [c for c in table_C.columns if c not in front]]
print("TABLE C - test-only error by input range (equal-count bins of the full da
display(table_C.style.format(precision=4))

def fig_bins(name, vars_, fname):
    r12_style()
    d = err_all[err_all.Model == name]
    nv = len(vars_)
    fig, axes = plt.subplots(2, nv, figsize=(5.8 * nv, 10), squeeze=False)
    for j, v in enumerate(vars_):
        labs = [l for l in bin_labels(bin_edges[v]) if (d[f"bin_{v}"] == l).any(
        tick = [f"{l}\n(n={int((d[f'bin_{v}'] == l).sum())}/{int(train_bin_count
        col = plt.cm.viridis(np.linspace(0.15, 0.85, max(len(labs), 2)))

```

```

ax = axes[0, j] # signed relative
vals = [100 * d[d[f"bin_{v}"] == 1].rel_err.values for l in labs]
bp = ax.boxplot(vals, positions=range(len(labs)), widths=0.55, patch_art
                showfliers=False, medianprops=dict(color="red", lw=2.2))
for k, patch in enumerate(bp["boxes"]):
    patch.set_facecolor(col[k]); patch.set_alpha(0.55); patch.set_linewi
for k, vv in enumerate(vals):
    ax.scatter(k + _rng.normal(0, 0.07, len(vv)), vv, s=20, c=[col[k]],
              edgecolors="black", linewidths=0.4, alpha=0.85, zorder=3)
ax.axhline(0, color="k", lw=1.2)
ax.set_xticks(range(len(labs))); ax.set_xticklabels(tick, rotation=30, h
ax.set_ylabel("Relative error (%)", fontsize=13); ax.set_title(f"Signed
ax.set_xlabel(f"{v} bin (n_test / n_train)", fontsize=12)

ax = axes[1, j] # MAE per bin with
mae = [d[d[f"bin_{v}"] == 1].abs_err.mean() for l in labs]
ci = [boot_ci(d[d[f"bin_{v}"] == 1].abs_err.values) for l in labs]
err = np.nan_to_num(np.array([[m - c[0], c[1] - m] for m, c in zip(mae,
ax.bar(range(len(labs)), mae, color=col[:len(labs)], edgecolor="black",
      alpha=0.75, yerr=err, capsize=6, error_kw=dict(lw=2))
ax.axhline(d.abs_err.mean(), color="red", ls="--", lw=1.6, label="overall
ax.set_xticks(range(len(labs))); ax.set_xticklabels(tick, rotation=30, h
ax.set_ylabel(f"MAE in {TL} units (95 % CI)", fontsize=13); ax.set_title
ax.set_xlabel(f"{v} bin (n_test / n_train)", fontsize=12)
if j == 0:
    ax.legend(fontsize=10)
for k, ax in enumerate(axes.ravel()):
    ax.tick_params(axis="y", labelsize=11); apply_journal_style(ax)
    for lab_ in ax.get_xticklabels():
        lab_.set_fontsize(9)
    panel_tag(ax, f"({'abcdefghij'[k]})")
fig.suptitle(f"{name} - test-only error across {'', ' '.join(vars_)} ranges", f
            fontweight="bold", y=1.0)
fig.tight_layout()
savefig_r12(fig, fname.format(name=name))
plt.show(); plt.close(fig)

_suppl = [v for v in BIN_VARS if v not in FIG_VARS_MAIN]
for _n in PLOT_MODELS:
    fig_bins(_n, FIG_VARS_MAIN, "{name}_R12_3_Errors_vs_Ca_Re_We.png")
    if _suppl:
        fig_bins(_n, _suppl, "{name}_R12_3b_Errors_vs_" + " ".join(v.replace(".",

```

8.6 Figure R12-4 — error maps in the (Ca, Re) and (Ca, We) planes

Grey = training points (sampling density), colour = test [relative error], red rings = flagged outliers.

```

In [ ]: # =====
# 8.6 ERROR MAPS IN THE (Ca, Re) AND (Ca, We) PLANES (FIGURE R12-4)
#       grey dots = training points (sampling density), coloured = test error,
#       red rings = flagged outliers (defined in 8.1)
# =====
def fig_maps(name):
    r12_style()
    d = err_all[err_all.Model == name]
    tr = data.loc[X_train.index]

```

```

vmax = np.quantile(100 * d.abs_rel_err, 0.98)
fig, axes = plt.subplots(1, len(MAP_PAIRS), figsize=(7.6 * len(MAP_PAIRS), 6
for ax, (a, b), tg in zip(axes[0], MAP_PAIRS, "abcdef"):
    ax.scatter(tr[a], tr[b], s=14, c="lightgray", edgecolors="gray", linewidth
               label=f"Train (n={len(tr)})", zorder=1)
    sc = ax.scatter(d[a], d[b], s=70, c=100 * d.abs_rel_err, cmap="YlOrRd",
                   edgecolors="black", linewidths=0.6, label="Test (colour
    o = d[d.is_outlier]
    if len(o):
        ax.scatter(o[a], o[b], s=210, facecolors="none", edgecolors="red", 1
                   label=f"Outliers (n={len(o)})", zorder=3)
    ax.set_xscale("log"); ax.set_yscale("log")
    ax.set_xlabel(a, fontsize=14); ax.set_ylabel(b, fontsize=14)
    ax.legend(fontsize=9, loc="best", frameon=True)
    cb = fig.colorbar(sc, ax=ax, pad=0.02, extend="max")
    cb.set_label("|Relative error| (%)", fontsize=12, fontweight="bold")
    ax.tick_params(axis="both", which="both", labelsiz=11)
    apply_journal_style(ax); panel_tag(ax, f"({tg})")
    ax.set_title(f"Error map: {a} - {b}", fontsize=13)
fig.suptitle(f"{name} - where are the errors, and how dense is the training
             fontweight="bold", y=1.02)
fig.tight_layout()
savefig_r12(fig, f"{name}_R12_4_Error_Maps.png")
plt.show(); plt.close(fig)

for _n in PLOT_MODELS:
    fig_maps(_n)

```

8.7 Figure R12-5 and Tables D, E — influential outliers

```

In [ ]: # =====
# 8.7 INFLUENTIAL OUTLIERS (TABLES D, E + FIGURE R12-5)
# * outlier : |robust z| > OUTLIER_Z (median/MAD based, robust to the outlie
#               computed within each film-thickness regime if OUTLIER_WITHIN_RE
# * influence: share of the total squared error + change of test RMSE / R2 whe
# * context : distance to the K nearest training points (sparse region?) and
#               the observed target among those neighbours
# =====
rows_D, rows_E = [], []
for name in base_models:
    d = err_all[err_all.Model == name].reset_index(drop=True)
    sse = (d.resid ** 2).sum()
    rmse_all = np.sqrt((d.resid ** 2).mean())
    r2_all = r2_score(d.y_true, d.y_pred)
    for i in d.index[d.is_outlier]:
        keep = d.index != i
        rows_D.append({
            "Model": name, "row_id": d.row_id[i], **{v: d.loc[i, v] for v in fea
            "y_true": d.y_true[i], "y_pred": d.y_pred[i], "resid": d.resid[i],
            "rel_err (%)": 100 * d.rel_err[i], "robust z": d.z_robust[i], "Regim
            "SSE share (%)": 100 * d.resid[i] ** 2 / sse,
            "ΔRMSE if removed": np.sqrt((d.resid[keep] ** 2).mean()) - rmse_all,
            "ΔR² if removed": r2_score(d.y_true[keep], d.y_pred[keep]) - r2_all,
            "kNN dist to train": d.knn_dist[i],
            "kNN dist percentile (test)": 100 * (d.knn_dist < d.knn_dist[i]).mea
            "nbr y min": d.nbr_y_min[i], "nbr y max": d.nbr_y_max[i],
        })
    # metrics with and without ALL flagged points
    ok = ~d.is_outlier

```

```

    for label, sel in (("all test points", np.ones(len(d), bool)), ("outliers re
        s = d[sel]
        rows_E.append({"Model": name, "Subset": label, "n": len(s),
            "R2": r2_score(s.y_true, s.y_pred), "RMSE": np.sqrt((s.re
            "MAE": s.abs_err.mean(), "MAPE (%)": 100 * s.abs_rel_err.
            "Bias": s.resid.mean()}))
table_D = pd.DataFrame(rows_D)
table_E = pd.DataFrame(rows_E)
if len(table_D):
    table_D["_abs_z"] = table_D["robust z"].abs()
    table_D = table_D.sort_values(["Model", "_abs_z"], ascending=[True, False]).
print(f"TABLE D - flagged outliers (|robust z| > {OUTLIER_Z}, basis = {OUTLIER_B
    f"{'within regime' if OUTLIER_WITHIN_REGIME else 'global'}))")
display(table_D.style.format(precision=4) if len(table_D) else "no outliers flag
print("\nTABLE E - test metrics with and without the flagged outliers")
display(table_E.style.format(precision=4))

def fig_outliers(name):
    r12_style()
    d = err_all[err_all.Model == name].reset_index(drop=True)
    o = d[d.is_outlier]
    fig, axes = plt.subplots(1, 3, figsize=(17, 5.6))

    ax = axes[0]
    # robust z vs obse
    ax.scatter(d.y_true, d.z_robust, s=38, c="#2980B9", edgecolors="black", line
    ax.scatter(o.y_true, o.z_robust, s=90, c="red", edgecolors="black", linewidth
    for _, r in o.iterrows():
        ax.annotate(str(int(r.row_id)), (r.y_true, r.z_robust), fontsize=8, xyte
            textcoords="offset points")
    for lv in (-OUTLIER_Z, OUTLIER_Z):
        ax.axhline(lv, color="red", ls="--", lw=1.5)
    ax.axhline(0, color="k", lw=1.2)
    ax.set_xlabel(f"Observed {TL}", fontsize=13)
    ax.set_ylabel(f"Robust z of {'residual' if OUTLIER_BASIS == 'resid' else 're
        + (" (within regime)" if OUTLIER_WITHIN_REGIME else ""), fonts
    ax.set_title(f"Outlier screening (|z| > {OUTLIER_Z:g}; labels = row index)",

    ax = axes[1]
    # error vs local t
    ax.scatter(d.knn_dist, 100 * d.abs_rel_err, s=38, c="#2980B9", edgecolors="b
    ax.scatter(o.knn_dist, 100 * o.abs_rel_err, s=90, c="red", edgecolors="black
    rho, p = stats.spearmanr(d.knn_dist, d.abs_rel_err)
    ax.text(0.04, 0.96, f"Spearman  $\rho$  = {rho:.2f}\n $p$  = {p:.2g}", transform=ax.tra
        va="top", fontweight="bold",
        bbox=dict(facecolor="white", edgecolor="black", linewidth=1.2, boxst
    ax.set_xlabel(f"Mean distance to {K_NEIGHBOURS} nearest training points (sta
    ax.set_ylabel("|Relative error| (%)", fontsize=13)
    ax.set_title("Error vs local training-data density", fontsize=12)

    ax = axes[2]
    # cumulative share
    sq = np.sort(d.resid.values ** 2)[::-1]
    cum = 100 * np.cumsum(sq) / sq.sum()
    xr = 100 * np.arange(1, len(sq) + 1) / len(sq)
    ax.plot(xr, cum, "-", color="#2980B9", lw=2.5)
    ax.plot([0, 100], [0, 100], "--", color="dimgray", lw=1.3, label="equal cont
    if len(o):
        f = 100 * len(o) / len(d)
        share = 100 * (o.resid ** 2).sum() / (d.resid ** 2).sum()
        ax.plot([f], [share], "o", color="red", ms=10, mec="black", zorder=3,
            label=f"{len(o)} outliers = {f:.1f} % of points\n> {share:.0f} %
    ax.set_xlabel("Share of test points, ranked by squared error (%)", fontsize=

```

```

ax.set_ylabel("Cumulative share of squared error (%)", fontsize=12)
ax.legend(fontsize=9.5, loc="lower right"); ax.set_title("Influence on the e

for ax, tg in zip(axes, "abc"):
    ax.tick_params(axis="both", labelsize=11); apply_journal_style(ax); pane
fig.suptitle(f"{name} - influential outliers (test set)", fontsize=16, fontw
fig.tight_layout()
savefig_r12(fig, f"{name}_R12_5_Outliers.png")
plt.show(); plt.close(fig)

for _n in PLOT_MODELS:
    fig_outliers(_n)

```

8.8 Figure R12-6 and Table F — error as dimensional film thickness vs experimental uncertainty

```

In [ ]: # =====
# 8.8 ERROR IN DIMENSIONAL FILM THICKNESS (TABLE F + FIGURE R12-6)
#      film thickness  $h = (\text{target}) \times R$  ->  $\text{error}_h = \text{error}_{\text{target}} \times R$ 
#      (the model error is in units of the target, so it scales linearly with R)
# =====
have_unc = (EXP_ABS_UNC_UM is not None) or (EXP_REL_UNC is not None)
if not have_unc:
    print("NOTE: EXP_ABS_UNC_UM / EXP_REL_UNC are None (8.1) -> the comparison w
          "uncertainty is skipped. Fill them in and re-run 8.8 to add it.")

def exp_unc_um(h_um):
    """Experimental uncertainty of the film thickness (um), at thickness h_um."""
    a = 0.0 if EXP_ABS_UNC_UM is None else EXP_ABS_UNC_UM
    r = 0.0 if EXP_REL_UNC is None else EXP_REL_UNC
    return np.sqrt(a ** 2 + (r * h_um) ** 2)

def verdict(ratio):
    # rule-of-thumb bands, adjust if you pre
    return "below" if ratio < 0.75 else ("comparable" if ratio <= 1.33 else "abo

rows = []
for name in base_models:
    d = err_all[err_all.Model == name]
    for R in CAPILLARY_RADII_MM:
        k = R * 1000.0 # target units ->
        for lab in ["All"] + REGIME_LABELS:
            s = d if lab == "All" else d[d.Regime == lab]
            if len(s) == 0:
                continue
            h_typ = np.median(s.y_true) * k
            row = {"Model": name, "R (mm)": R, "Regime": lab, "n": len(s),
                  "Typical h (um)": h_typ,
                  "Bias (um)": s.resid.mean() * k,
                  "MAE (um)": s.abs_err.mean() * k,
                  "RMSE (um)": np.sqrt((s.resid ** 2).mean()) * k,
                  "P95 |err| (um)": np.quantile(s.abs_err, 0.95) * k}
            if have_unc:
                u = exp_unc_um(h_typ)
                row["U_exp (um)"] = u
                row["RMSE / U_exp"] = row["RMSE (um)"] / u if u > 0 else np.nan
                row["Verdict (RMSE vs U_exp)"] = verdict(row["RMSE / U_exp"]) if
            rows.append(row)
table_F = pd.DataFrame(rows)

```

```

print("TABLE F - test-only prediction error expressed as film thickness ( $\mu\text{m}$ )")
display(table_F.style.format(precision=3))

def fig_dim(name):
    r12_style()
    t = table_F[table_F.Model == name]
    Rs = list(CAPILLARY_RADII_MM)
    fig, axes = plt.subplots(1, len(Rs), figsize=(5.8 * len(Rs), 5.6), squeeze=False)
    labs = ["A11"] + REGIME_LABELS
    w = 0.26
    for ax, R, tg in zip(axes[0], Rs, "abcdefgh"):
        s = t[t["R (mm)"] == R].set_index("Regime").reindex(labs)
        x = np.arange(len(labs))
        ax.bar(x - w, s["MAE ( $\mu\text{m}$ )"], w, color="#5DADE2", edgecolor="black", line)
        ax.bar(x, s["RMSE ( $\mu\text{m}$ )"], w, color="#2E86C1", edgecolor="black", line)
        ax.bar(x + w, s["P95 |err| ( $\mu\text{m}$ )"], w, color="#1B4F72", edgecolor="black")
        if have_unc:
            ax.plot(x, s["U_exp ( $\mu\text{m}$ )"], "D", color="red", ms=10, mec="black", zorder=2,
                    label="Experimental uncertainty")
        ax.set_xticks(x)
        ax.set_xticklabels([f"{l}\nh={s.loc[l, 'Typical h ( $\mu\text{m}$ )']:.0f}  $\mu\text{m}$ " for l in labs])
        ax.set_ylabel("Film-thickness error ( $\mu\text{m}$ )", fontsize=13)
        ax.set_title(f"R = {R:g} mm", fontsize=14)
        ax.tick_params(axis="y", labelsize=11); apply_journal_style(ax); panel_title(ax, tg)
        for lab_ in ax.get_xticklabels():
            lab_.set_fontsize(10)
        if tg == "a":
            ax.legend(fontsize=9.5, loc="upper left")
    fig.suptitle(f"{name} - prediction error as dimensional film thickness"
                + (" vs experimental uncertainty" if have_unc else ""), fontsize=12)
    fig.tight_layout()
    savefig_r12(fig, f"{name}_R12_6_Dimensional_Uncertainty.png")
    plt.show(); plt.close(fig)

for _n in PLOT_MODELS:
    fig_dim(_n)

```

8.9 Save all tables

```

In [ ]: # =====
# 8.9 SAVE ALL TABLES OF THIS SECTION TO ONE EXCEL WORKBOOK
# =====
xlsx_path = os.path.join(R12_DIR, "Reviewer12_Error_Analysis_Tables.xlsx")
tidy = err_all.copy()
for c in tidy.columns:                                # categoricals -> plain
    if str(tidy[c].dtype) == "category":
        tidy[c] = tidy[c].astype(str)
with pd.ExcelWriter(xlsx_path, engine="openpyxl") as xw:
    table_A.reset_index().to_excel(xw, sheet_name="A_Overall_test_only", index=False)
    table_B.reset_index().to_excel(xw, sheet_name="B_By_regime", index=False)
    table_C.reset_index().to_excel(xw, sheet_name="C_By_Ca_Re_We_bin", index=False)
    table_D.to_excel(xw, sheet_name="D_Outliers", index=False)
    table_E.to_excel(xw, sheet_name="E_With_without_outliers", index=False)
    table_F.to_excel(xw, sheet_name="F_Dimensional_um", index=False)
    tidy.to_excel(xw, sheet_name="G_Test_predictions", index=False)
print("Saved tables :", xlsx_path)
print("Saved figures :", R12_DIR)

```

```

In [ ]: # =====
# 9. REVISED TABLE 7 (train / CV mean  $\pm$  SD / test with 95 % CI)
# =====

from sklearn.base import clone
from sklearn.model_selection import RepeatedKFold, GroupKFold, cross_validate
from scipy import stats

# ----- settings -----
T7_N_SPLITS = 5          # folds
T7_N_REPEATS = 3         # repeats of the k-fold split (SD is taken over all n
T7_N_BOOT = 2000         # bootstrap resamples of the test set
T7_SEED = 42
RUN_GROUP_CV = True      # also run CV that holds out whole experimental serie
GROUP_COLS = ["Bo", "V.R", "D.R"] # columns that identify one experimental
T7_DIR = os.path.join(OUTPUT_DIR, "Table7_revised")
os.makedirs(T7_DIR, exist_ok=True)

SCORING = {"r2": "r2",
           "rmse": "neg_root_mean_squared_error",
           "mape": "neg_mean_absolute_percentage_error"}

def _metrics(yt, yp):
    yt, yp = np.asarray(yt, float), np.asarray(yp, float)
    return (r2_score(yt, yp), np.sqrt(np.mean((yp - yt) ** 2)), 100 * np.mean(np

def _decimals(sd):
    """decimals so that the SD has 1 significant figure (2 if it starts with a 1
    if not np.isfinite(sd) or sd <= 0:
        return 3
    e = int(np.floor(np.log10(sd)))
    lead = int(round(sd / 10 ** e)) if sd / 10 ** e < 9.5 else 10
    return max(0, -e + (1 if lead == 1 else 0))

def fmt_pm(mean, sd):
    d = _decimals(sd)
    return f"{mean:.{d}f}  $\pm$  {sd:.{d}f}"

def fmt_ci(val, lo, hi, d):
    return f"{val:.{d}f} [{lo:.{d}f}, {hi:.{d}f}]"

def tuned(name):
    return clone(base_models[name]).set_params(**best_params_all[name])

# ---- 1) train / test metrics and bootstrap CI of the TEST metrics -----
rng = np.random.default_rng(T7_SEED)
yt_arr = y_test.values
boot_idx = rng.integers(0, len(yt_arr), size=(T7_N_BOOT, len(yt_arr)))

point = {}
for name in base_models:
    m = tuned(name).fit(X_train, y_train)
    tr = _metrics(y_train.values, m.predict(X_train))
    pt = m.predict(X_test)
    te = _metrics(yt_arr, pt)
    bs = np.array([_metrics(yt_arr[i], pt[i]) for i in boot_idx]) # (n_boot
    lo, hi = np.percentile(bs, [2.5, 97.5], axis=0)
    point[name] = dict(train=tr, test=te, lo=lo, hi=hi)

# ---- 2) repeated k-fold CV inside the training set (same folds for every model

```

```

rkf = RepeatedKfold(n_splits=T7_N_SPLITS, n_repeats=T7_N_REPEATS, random_state=T
cv_scores = {}
for name in base_models:
    r = cross_validate(tuned(name), X_train, y_train, cv=rkf, scoring=SCORING, n
    cv_scores[name] = {"r2": r["test_r2"], "rmse": -r["test_rmse"], "mape": -100
J = T7_N_SPLITS * T7_N_REPEATS

# ---- 3) assemble the revised Table 7 -----
# fixed display precision for POINT estimates (Train, Test) – independent of the
# CV fold-to-fold SD, so a real value like 0.9973 or 0.9989 never gets rounded
# down to a misleading "1.00".
DEC_PT = {"r2": 3, "rmse": 4, "mape": 2}

rows = []
for name in base_models:
    row = {("", "Model"): name}
    for k, (key, lab) in enumerate([("r2", "R2"), ("rmse", "RMSE"), ("mape", "MA
    v = cv_scores[name][key]
    mu, sd = v.mean(), v.std(ddof=1)
    d_pt = DEC_PT[key] # Train / Test always get this fixed preci
    row[(lab, "Train")] = f"{point[name]['train'][k]:.{d_pt}f}"
    row[(lab, f"CV ({T7_N_SPLITS}-fold x{T7_N_REPEATS}), mean ± SD")] = fmt_
    row[(lab, "Test [95 % CI]")] = fmt_ci(point[name]['test'][k], point[name
    point[name]['hi'][k], d_pt)

    rows.append(row)
table7 = pd.DataFrame(rows)
table7.columns = pd.MultiIndex.from_tuples(table7.columns)
table7 = table7.set_index("", "Model"); table7.index.name = "Model"
print("REVISED TABLE 7 (RMSE in target units; Train/Test shown to fixed precisi
display(table7)

# ---- 4) paired comparison with the best model (same folds, Nadeau-Bengio) ----
best = max(base_models, key=lambda n: cv_scores[n]["r2"].mean())
ratio = 1.0 / (T7_N_SPLITS - 1) # n_test / n_train of one f
rows = []
for name in base_models:
    if name == best:
        continue
    r = {"Model": name, "vs": best}
    for key, lab, sign in [("r2", "R2", +1), ("rmse", "RMSE", -1)]:
        d = sign * (cv_scores[best][key] - cv_scores[name][key]) # > 0 => bes
        se = np.sqrt((1.0 / J + ratio) * d.var(ddof=1))
        t = d.mean() / se if se > 0 else np.nan
        p = 2 * stats.t.sf(abs(t), df=J - 1)
        tc = stats.t.ppf(0.975, df=J - 1)
        r[f"Δ{lab} (best - model)"] = sign * d.mean()
        r[f"Δ{lab} 95 % CI"] = f"[{sign*(d.mean()-tc*se):.4f}, {sign*(d.mean()+t
        else f"[{sign*(d.mean()+tc*se):.4f}, {sign*(d.meas
        r[f"p ({lab})"] = p
    rows.append(r)
paired = pd.DataFrame(rows)
print(f"\nPAIRED COMPARISON with the best model by CV R2 = {best} "
    f"(same {J} folds; Nadeau-Bengio corrected t-test)")
print("p > 0.05 => the difference is NOT distinguishable from fold-to-fold var
display(paired.style.format({"p (R2)": "{:.3f}", "p (RMSE)": "{:.3f}"}, precisio

# ---- 5) optional: group-wise CV (hold out whole experimental series) -----
group_table = None
if RUN_GROUP_CV:
    key = data.loc[X_train.index, GROUP_COLS].round(8).astype(str).agg("|".join,

```

```

gid = pd.factorize(key)[0]
n_groups = len(np.unique(gid))
k_g = min(5, n_groups)
print(f"\nGroup-wise CV: {n_groups} experimental series in the training set,
if k_g >= 3:
    rows = []
    for name in base_models:
        r = cross_validate(tuned(name), X_train, y_train, cv=GroupKFold(n_sp
            scoring=SCORING, n_jobs=1)
        vals = {"r2": r["test_r2"], "rmse": -r["test_rmse"], "mape": -100 *
        row = {"Model": name}
        for key_, lab in [("r2", "R2"), ("rmse", "RMSE"), ("mape", "MAPE (%)
            row[f"{lab} ({k_g}-fold group CV, mean ± SD)"] = fmt_pm(vals[ke
        rows.append(row)
    group_table = pd.DataFrame(rows).set_index("Model")
    print("GROUP-WISE CV: every fold predicts series that were NOT seen in t
        "(stricter than random folds; compare with the CV column above)")
    display(group_table)
else:
    print("Too few series for group-wise CV; skipped.")

# ---- 6) save -----
xl = os.path.join(T7_DIR, "Table7_revised.xlsx")
with pd.ExcelWriter(xl, engine="openpyxl") as xw:
    t7 = table7.copy(); t7.columns = [" | ".join(c).strip(" |") for c in t7.colu
    t7.reset_index().to_excel(xw, sheet_name="Table7_revised", index=False)
    paired.to_excel(xw, sheet_name="Paired_vs_best", index=False)
    if group_table is not None:
        group_table.reset_index().to_excel(xw, sheet_name="Group_CV", index=False)
        raw = pd.DataFrame({f"{n} | {k}": cv_scores[n][k] for n in base_models for k
        raw.index.name = "fold"; raw.to_excel(xw, sheet_name="CV_fold_scores_raw")
print("\nSaved:", xl)

```